UNIVERSIDAD DE LOS ANDES
ISIS-1226 · DISEÑO Y PROGRAMACIÓN ORIENTADA A OBJETOS

INFORME DE DISEÑO

Sistema de analítica deportiva del Club Deportivo Séneca

Proyecto 1 — Entrega 2

INTEGRANTES

Alejandro Torres Carvajal, Samuel Soto y Tomas HIguera.

Bogotá, 2026

TABLA DE CONTENIDO

01 DOCUMENTO DE DISEÑO ..... p. 4

1. Introducción
2. Objetivos
3. Alcance del proyecto
4. Arquitectura general
5. Modelos y diagramas que acompañan este diseño
6. Documentos complementarios
7. Estado de la implementación
8. División del trabajo en el grupo

02 DECISIONES DE DISEÑO ..... p. 7

1. Propósito de este documento
2. El administrador hereda de Persona y no de Afiliado
3. Fútbol, baloncesto y voleibol no son subclases
4. Las clases asociación
5. PartidoCompetencia se relaciona con Nómina y no con Equipo
6. La estadística se relaciona con Afiliado y no con AfiliadoConjunto
7. ProductoCafeteria se separó en Bebida y Snack
8. Club es una fachada y no un Singleton
9. Categoría y Nivel son enumeraciones y no texto
10. Uso de java.time para fechas y horas
11. Limitaciones asumidas conscientemente

03 PATRONES DE DISEÑO APLICADOS ..... p. 11

1. Criterio de selección
2. Strategy
3. Factory Method

4. Template Method
5. DAO
6. Fachada
7. Resumen

04 MODELO DE PERSISTENCIA Y MANEJO DE ERRORES ..... p. 14

1. Requisitos de persistencia
2. Mecanismo principal: archivos planos
3. Mecanismo complementario: serialización binaria
4. La interfaz que los desacopla
5. Manejo de errores

05 TRAZABILIDAD DE REQUERIMIENTOS ..... p. 16

1. Propósito
2. Requerimientos funcionales del administrador
3. Requerimientos del afiliado deportista de conjunto
4. Requerimientos del afiliado deportista individual
5. Requerimientos del entrenador y del fisioterapeuta
6. Requerimientos comunes a afiliados y empleados
7. Restricciones del proyecto
8. Cobertura

06 CAMBIOS RESPECTO A LA ENTREGA 1 ..... p. 19

1. Por qué hubo cambios
2. Correcciones señaladas en la retroalimentación
3. Cambios surgidos al pasar al diseño
4. Lo que no cambió

07 MANUAL DE INSTALACIÓN Y EJECUCIÓN ..... p. 21

1. Requisitos
2. Importar el proyecto en Eclipse
3. Compilar desde la línea de comandos
4. Estructura de carpetas
5. Sobre la ejecución
6. Los archivos de datos
7. Dónde empezar a leer el código

08 ANEXO DE USO DE INTELIGENCIA ARTIFICIAL GENERATIVA ..... p. 23 1.

Alcance del uso
2. Herramientas utilizadas
3. Colección de prompts
4. Revisión del grupo

DOCUMENTO DE DISEÑO

Sistema de analítica deportiva del Club Deportivo Séneca

Proyecto 1 — Entrega 2

1. INTRODUCCIÓN

El Club Deportivo Séneca es la rama deportiva de una institución universitaria y tiene un propósito doble: formar a sus integrantes a lo largo de distintas etapas de edad y representar a la institución en los circuitos deportivos universitarios de la ciudad y del país. Su operación combina dos frentes que hoy se manejan por separado y sin un registro unificado.

El primero es el frente deportivo. El club ofrece tres deportes de conjunto —fútbol, baloncesto y voleibol— organizados por categorías de edad, y tres academias de disciplinas individuales de raqueta: tenis, pádel y tenis de mesa. Alrededor de ellos giran los entrenamientos, los partidos internos, las competencias externas y las estadísticas de cada jugador. El segundo es el frente comercial y administrativo: una tienda de artículos deportivos, una cafetería para los afiliados, el inventario de ambas y los turnos del personal.

La necesidad que atiende el proyecto es llevar un control ordenado de quién juega, en qué equipo, bajo qué reglas y con qué resultados, y hacerlo de forma que las reglas del club no queden solo escritas en un reglamento sino incorporadas en el sistema. Varias de esas reglas no son opcionales: un jugador suspendido no puede alinearse, una solicitud de cambio de turno no puede aprobarse si deja al club sin la cobertura mínima de personal, y una bebida caliente no puede despacharse a alguien que esté dentro de una cancha techada.

La solución implementada es una aplicación en Java organizada en capas, donde el modelo del dominio está separado del mecanismo de persistencia y de la forma en que se presentan los datos. El diseño se apoya en cinco patrones de diseño, cada uno escogido para resolver un problema concreto del caso y no por completitud. La información se guarda de forma persistente en archivos, en una carpeta distinta a la del código fuente.

2. OBJETIVOS

2.1 Objetivo general

Diseñar e implementar en Java el modelo de un sistema de analítica deportiva para el Club Deportivo Séneca, de modo que las entidades del dominio, sus relaciones y las reglas de negocio identificadas en la Entrega 1 queden representadas en una estructura de clases coherente, extensible y defendible.

2.2 Objetivos específicos

Traducir el modelo de dominio de la Entrega 1 a un modelo de diseño, incorporando las correcciones señaladas en la retroalimentación y las decisiones que el análisis dejó pendientes.

Aplicar herencia, polimorfismo e interfaces donde el problema lo justifique, evitando jerarquías que no agreguen estructura. Incorporar patrones de diseño que resuelvan problemas concretos del caso, dejando documentado qué ocurriría si no se hubieran aplicado.

Implementar un mecanismo de persistencia en archivos que sea legible, editable a mano y desacoplado del modelo.

Dejar el sistema preparado para que en la entrega siguiente se construyan los programas de prueba y las pruebas unitarias sin tener que modificar el modelo.

3. ALCANCE DEL PROYECTO

El sistema cubre la gestión deportiva y comercial del club para tres tipos de usuario: afiliados, empleados y un administrador. Los afiliados se dividen según practiquen un deporte de conjunto o una disciplina individual, y los empleados según sean entrenadores o fisioterapeutas, porque el club exige mínimos de personal distintos para cada uno.

3.1 Funcionalidades incluidas

El modelo soporta la inscripción de jugadores en equipos con todas sus validaciones, el registro de partidos internos y de competencia con sus estadísticas individuales, la generación automática de sanciones por acumulación de tarjetas, la inscripción de equipos en competencias externas con dos formatos de juego distintos, las reservas de instalaciones con control de cupo simultáneo, los partidos de práctica con actualización del puntaje de ranking, las sesiones de entrenamiento con registro de asistencia y evaluación, la gestión de turnos con validación de cobertura mínima, y las ventas de tienda y cafetería con impuestos, descuentos de empleado y puntos de fidelidad.

3.2 Fuera del alcance de esta entrega

Esta entrega no incluye programas de consola ni pruebas unitarias, que corresponden a la entrega siguiente del proyecto. Tampoco
incluye la interfaz de interacción con el usuario, que el enunciado difiere al Proyecto 2, ni las visualizaciones de rendimiento y evolución de rankings, previstas para el Proyecto 3. El modelo sí deja preparado el terreno para las tres cosas: el historial de ranking ya se almacena y la fachada ya expone las operaciones que la interfaz necesitará.

4. ARQUITECTURA GENERAL

El código está organizado en capas para que el modelo del dominio no dependa de cómo se guardan los datos ni de cómo se muestran. Esta separación tiene una consecuencia práctica: si mañana se cambia el mecanismo de persistencia, solo se toca un paquete. Si en cambio las clases del modelo tuvieran código de lectura y escritura de archivos adentro, cambiar el formato obligaría a tocar todo el modelo.

4.1 Organización en paquetes

La raíz de todos los paquetes es uniandes.dpoo.cds. Debajo de ella, el modelo del dominio se subdivide por área del

problema y las capas técnicas tienen sus propios paquetes.

Paquete	Contenido
modelo.personas	La jerarquía de Persona: afiliados, empleados y administrador.
modelo.deportes	Equipos, inscripciones, partidos, estadísticas y sanciones.
modelo.competencias	Competencias externas, nóminas y formatos de juego.
modelo.academias	Disciplinas individuales, instalaciones y reservas.
modelo.entrenamiento	Sesiones, convocatorias y turnos del personal.
modelo.tienda	Productos y ventas de tienda y cafetería.
reglas	Las reglas particulares de cada deporte y de cada disciplina.
logica	La clase Club, que actúa como fachada del sistema.
persistencia	Lectura y escritura de los archivos de datos.
excepciones	Los errores propios del dominio.

4.2 Las jerarquías de herencia

El modelo tiene cinco jerarquías. En todas ellas, la clase de arriba concentra lo que las demás comparten y cada clase de abajo agrega únicamente lo que la distingue. Las clases abstractas existen porque nadie es «eso» a secas: en el club nadie está registrado simplemente como persona, y nadie vende simplemente un producto.

```
Persona (abstracta) Partido (abstracta)
    |-- Afiliado (abstracta) |-- PartidoInterno
    | |-- AfiliadoConjunto |-- PartidoCompetencia
    | |-- AfiliadoIndividual |-- PartidoPractica
    |-- Empleado (abstracta)
    | |-- Entrenador Estadistica (abstracta)
    | |-- Fisioterapeuta |-- EstadisticaFutbol
    |-- Administrador |-- EstadisticaBaloncesto
                                        |-- EstadisticaVoleibol
Producto (abstracta) |-- EstadisticaRaqueta
    |-- ArticuloDeportivo |-- EstadisticaTenisMesa
    |-- ProductoCafeteria (abstracta)
            |-- Bebida Venta (abstracta)
            |-- Snack |-- VentaTienda
                                        |-- VentaCafeteria
```

4.3 Las interfaces

El diseño define cuatro interfaces. ReglasDeporte y ReglasDisciplina encapsulan las reglas particulares de cada deporte y de cada disciplina de raqueta. FormatoCompetencia encapsula la forma en que cada circuito calcula su clasificación. RepositorioClub define cómo se guarda y se recupera la información, con dos implementaciones intercambiables.

4.4 Las clases asociación

Hay cinco clases cuya razón de existir es guardar datos que no pertenecen a ninguna de las dos clases que relacionan. El caso más claro es el número de camiseta: no es del jugador, porque el mismo jugador puede llevar la diez en fútbol y la siete en voleibol, y tampoco es del equipo, porque el equipo tiene muchas camisetas. Pertenece a la inscripción, es decir, a la unión de ese jugador con ese equipo.

Las cinco son InscripcionEquipo, que une a un afiliado de conjunto con un equipo; InscripcionAcademia, que une a un afiliado individual con una disciplina y guarda su nivel y su ranking; Nomina, que une a un equipo con una competencia y guarda los jugadores habilitados y el puntaje del torneo; Convocatoria, que une a un afiliado con una sesión de entrenamiento y guarda su asistencia y su evaluación; e ItemVenta, que une a un producto con una venta y guarda la cantidad y el precio al momento de la compra.

5. MODELOS Y DIAGRAMAS QUE ACOMPAÑAN ESTE DISEÑO

El diseño se documenta en tres diagramas, acompañados de los documentos complementarios que se listan en la sección siguiente.

5.1 Diagrama de clases de diseño

Archivo DIAGRAMA_DE_CLASES_DISENO_CDS_ENTREGA2.drawio. Incluye las setenta y dos clases del sistema con sus atributos, sus métodos y sus relaciones, agrupadas en bloques por paquete. Fue generado a partir del código fuente, de modo que el diagrama y la implementación no pueden desincronizarse: cualquier cambio en el código se refleja al regenerarlo.

5.2 Diagrama de clases de alto nivel

Presenta las mismas clases y relaciones pero sin atributos ni métodos. Su propósito es permitir entender la estructura de relaciones sin el ruido del detalle, que es difícil de ver en el diagrama completo.

5.3 Diagramas de secuencia

Documentan las operaciones críticas del sistema, entendiendo por críticas aquellas que involucran a varias clases y en las que el orden de los pasos cambia el resultado. Son tres: la aprobación de un cambio de turno, el registro de un partido interno con sus sanciones, y la liquidación de una venta.

6. DOCUMENTOS COMPLEMENTARIOS

El enunciado advierte que un documento que incluya únicamente los tres diagramas anteriores será considerado insuficiente. Por esa razón el diseño se documenta en los siguientes archivos, cada uno dedicado a un aspecto:

1. Documento de diseño (este documento): contexto, objetivos, alcance y arquitectura general.
2. Decisiones de diseño: las decisiones discutibles del modelo, con las alternativas que se consideraron y se descartaron. Incluye la justificación de los dos elementos del lenguaje que no se trabajaron en clase: las enumeraciones y el paquete java.time. 3. Patrones de diseño aplicados: los cinco patrones, con el problema que resuelve cada uno.
4. Modelo de persistencia y manejo de errores: estructura de los archivos y jerarquía de excepciones.
5. Trazabilidad de requerimientos: cada requerimiento funcional y cada restricción contra la clase y el método que los implementa. 6. Cambios respecto a la Entrega 1: qué se corrigió del modelo de dominio y por qué.
7. Manual de instalación y ejecución.
8. Anexo de uso de inteligencia artificial generativa.

7. ESTADO DE LA IMPLEMENTACIÓN

El proyecto compila sin errores y contiene setenta y dos clases distribuidas en los diez paquetes descritos. No contiene todavía ninguna clase con método main, porque los programas ejecutables y las pruebas corresponden a la entrega siguiente. La carpeta de datos incluye archivos de ejemplo que muestran el formato de persistencia y que servirán para alimentar esas pruebas.

8. DIVISIÓN DEL TRABAJO EN EL GRUPO

El reparto que se indicó en la Entrega 1 cambió al empezar esta entrega, porque el diagrama de clases de diseño quedó atado a la implementación: se genera a partir del código fuente, de modo que quien escribió las clases es quien tiene que mantenerlo. Las tareas de diagramación restantes pasaron entonces a Samuel, y la consolidación de la documentación a Tomás. Este es el reparto vigente para la Entrega 2:

Integrante	Responsabilidad en esta entrega
Mateo Pulido	Implementación completa del modelo en Eclipse, es decir las setenta y dos clases con su jerarquía, relaciones, atributos y métodos; el diagrama de clases de diseño, que se genera a partir de ese código; y la redacción de los documentos de diseño, decisiones, patrones, persistencia, trazabilidad, cambios y manual.
Samuel	Los dos diagramas restantes del entregable: el diagrama de objetos de un escenario representativo y los diagramas de secuencia de las tres operaciones críticas.
Tomás	Consolidación y revisión de la documentación: verificar que los nombres de clases y métodos coincidan entre todos los documentos y el código, armar el documento del anexo de uso de inteligencia artificial generativa con los aportes de los tres, y preparar el guion de la sustentación.

Se deja constancia del cambio porque el reparto documentado en la Entrega 1 asignaba la diagramación y la documentación al contrario de como quedaron.

DECISIONES DE DISEÑO

Sistema de analítica deportiva del Club Deportivo Séneca
Proyecto 1 — Entrega 2

1. PROPÓSITO DE ESTE DOCUMENTO

Un diagrama muestra cómo quedó el modelo, pero no por qué quedó así. Este documento recoge las ocho decisiones del diseño que podrían haberse tomado de otra manera, y para cada una deja constancia de tres cosas: cuál era el problema concreto del caso, qué alternativa se consideró y se descartó, y qué consecuencia tendría no haberla tomado.

Se incluyen también las limitaciones que el diseño asume de forma consciente. Reconocer el límite de una decisión es parte de sustentarla.

2. EL ADMINISTRADOR HEREDA DE PERSONA Y NO DE AFILIADO

2.1 El problema

El enunciado establece que el administrador no puede inscribirse a partidos ni reservar instalaciones para uso personal. Es la restricción que lo distingue de cualquier otro usuario del sistema.

2.2 La alternativa descartada

Hacer que Administrador heredara de Afiliado, aprovechando que comparte con él buena parte de los datos, y validar con una condición en cada método que intentara reservar o inscribir.

2.3 La decisión

Administrador hereda directamente de Persona. El método Club.crearReserva recibe un AfiliadoIndividual como parámetro, y un administrador no lo es.

2.4 Qué pasaría si no se hubiera hecho así

La restricción dependería de que alguien se acuerde de escribir la validación en todos los puntos de entrada, y de que nadie la borre después. Con la decisión tomada, la restricción la hace cumplir el compilador: escribir club.crearReserva(codigo, administrador, ...) impide que el proyecto compile. Es la diferencia entre ponerle candado a una puerta y no ponerle puerta al cuarto donde nadie debe entrar.

3. FÚTBOL, BALONCESTO Y VOLEIBOL NO SON SUBCLASES

3.1 El problema

Los tres deportes de conjunto comparten exactamente la misma estructura —equipos, jugadores inscritos, partidos, estadísticas— pero se diferencian en sus reglas: el mínimo de jugadores para presentarse a un partido, los puntos que otorga una victoria, si existe el empate, si se manejan tarjetas y si el marcador se lleva por sets.

3.2 La alternativa descartada

Crear tres subclases de deporte llamadas Futbol, Baloncesto y Voleibol. Quedarían prácticamente vacías, porque no agregan estructura nueva sino valores distintos para los mismos campos.

3.3 La decisión

Una interfaz ReglasDeporte con tres implementaciones, una por deporte. La clase Equipo guarda una referencia a sus reglas y les pregunta cuando necesita saber algo.

3.4 Qué pasaría si no se hubiera hecho así

Las clases Equipo y Partido se llenarían de condicionales del tipo «si el deporte es fútbol entonces el mínimo es siete, si no, si es baloncesto entonces es cinco», repetidos en cada método que dependa del deporte. Agregar un cuarto deporte obligaría a buscar y modificar todos esos condicionales, con el riesgo de olvidar alguno. Con las reglas separadas basta con escribir una clase nueva sin tocar nada existente.

3.5 Dónde sí se usó herencia

En las estadísticas. Ahí la diferencia sí es estructural: el fútbol registra goles, asistencias y tarjetas; el baloncesto puntos y rebotes; el voleibol aces y bloqueos. No son los mismos datos con valores distintos, son datos distintos, y por eso cada deporte tiene su propia clase de estadística.

4. LAS CLASES ASOCIACIÓN

4.1 El problema

Hay datos que no pertenecen a ninguna de las dos clases que relacionan. El número de camiseta no es del jugador, porque el mismo jugador puede llevar la diez en fútbol y la siete en voleibol; tampoco es del equipo, porque el equipo tiene

muchas camisetas. El nivel de juego no es del afiliado, porque puede ser avanzado en tenis y principiante en pádel al mismo tiempo; tampoco es de la disciplina.

4.2 La decisión

Cinco clases cuya razón de existir es guardar esos datos intermedios: InscripcionEquipo, InscripcionAcademia, Nomina, Convocatoria e ItemVenta.

4.3 El caso de ItemVenta

ItemVenta guarda el precio unitario en lugar de consultárselo al producto. La razón es que si mañana sube el precio de un artículo, las facturas emitidas antes deben seguir mostrando lo que efectivamente se cobró ese día. Si el precio se consultara al producto en el momento de imprimir, el histórico de ventas cambiaría solo.

4.4 Qué pasaría si no se hubiera hecho así

Habría que elegir arbitrariamente una de las dos puntas para guardar el dato, y esa elección rompería en el primer caso real: un jugador en dos deportes, un afiliado en dos academias o un producto que cambia de precio.

5. PARTIDOCOMPETENCIA SE RELACIONA CON NOMINA Y NO CON EQUIPO

5.1 El problema

El enunciado establece que solo los jugadores registrados en la nómina de una competencia pueden disputar sus partidos oficiales. Un equipo puede estar inscrito en dos circuitos distintos con nóminas diferentes.

5.2 La decisión

El partido de competencia apunta a la Nomina, que es la inscripción de ese equipo en ese torneo, y no directamente al equipo.

5.3 Qué pasaría si no se hubiera hecho así

La regla de la nómina quedaría escrita en el documento pero no representada en el modelo. Nada en la estructura impediría alinear a un jugador que no está inscrito en ese torneo, y la validación dependería otra vez de una condición que alguien tendría que acordarse de escribir.

6. LA ESTADÍSTICA SE RELACIONA CON AFILIADO Y NO CON AFILIADOCONJUNTO

6.1 El problema

Esta fue una de las correcciones señaladas en la retroalimentación de la Entrega 1. En el modelo de dominio, la estadística solo se conectaba con el afiliado de deporte de conjunto, de modo que los partidos de práctica de las academias individuales quedaban sin un lugar donde registrar sus datos, pese a que el enunciado pide guardar sets, games, aces y errores no forzados.

6.2 La decisión

Estadistica se relaciona con Afiliado, que es la clase padre. Como PartidoPractica también es un Partido, los dos casos quedan cubiertos por la misma jerarquía sin duplicar ninguna relación.

6.3 Lo que además se corrigió

La clase original guardaba los datos en campos llamados valorPrincipal y valorSecundario, que no decían qué medían. Se reemplazó por una jerarquía donde cada deporte nombra lo suyo: la estadística de fútbol habla de goles y de tarjetas, la de baloncesto de puntos y rebotes.

7. PRODUCTOCAFETERIA SE SEPARÓ EN BEBIDA Y SNACK

7.1 El problema

En la Entrega 1 la cafetería era una sola clase con los campos tipoProducto, temperatura y alergenos.

7.2 La decisión

Dos subclases de ProductoCafeteria: Bebida, que sabe si es caliente o fría, y Snack, que declara sus alérgenos.

7.3 Qué pasaría si no se hubiera hecho así

Con una sola clase, la mitad de los atributos quedaría siempre vacía: una bebida no declara alérgenos y un snack no tiene temperatura. Además, la restricción de seguridad sobre las bebidas calientes se expresa naturalmente como un método de Bebida; en una clase mixta, ese método tendría que devolver un valor arbitrario cuando el producto fuera un snack.

8. CLUB ES UNA FACHADA Y NO UN SINGLETON

8.1 El problema

Hace falta un punto de entrada que coordine las operaciones que involucran a varias clases, como registrar un partido o aprobar un cambio de turno.

8.2 La alternativa descartada

Implementar Club como Singleton, con una única instancia accesible globalmente.

8.3 La decisión

Club es una fachada corriente: se construye con new y se le entrega a quien la necesite.

8.4 Por qué se descartó el Singleton

Por dos razones. La primera es práctica: obligaría a que exista un solo club durante toda la ejecución, lo que complica las pruebas de la entrega siguiente, donde cada prueba necesita partir de un estado limpio. La segunda es de diseño: el Singleton esconde la dependencia, porque cualquier clase podría llamar a Club.getInstancia() sin que esa dependencia aparezca en su firma ni en el diagrama.

8.5 Qué va dentro de la fachada y qué no

En Club solo están las reglas que cruzan varias clases, como la cobertura mínima de personal o el cupo de instalaciones simultáneas por disciplina. Las reglas que pertenecen a un solo objeto viven en ese objeto: que un equipo tenga cupo disponible lo sabe el equipo, no el club.

9. CATEGORÍA Y NIVEL SON ENUMERACIONES Y NO TEXTO

9.1 El problema

En la Entrega 1, la categoría de un equipo y el nivel de un afiliado eran cadenas de texto. Pero de ambos dependen validaciones: la categoría define un rango de edad y el nivel define contra quién puede jugar un afiliado.

9.2 La decisión

Se convirtieron en enumeraciones. Categoria lleva su rango de edad y responde si admite una edad dada. Nivel lleva un orden y responde si otro nivel es adyacente al suyo.

9.3 Qué pasaría si no se hubiera hecho así

Los rangos de edad habría que repetirlos en cada validación, y la regla de los niveles adyacentes sería imposible de expresar sin convertir el texto a un número en cada comparación. Además, un error de escritura como «Principiante» con minúscula inicial pasaría inadvertido hasta que una validación fallara en silencio.

9.4 Aclaración sobre el uso de enum

La palabra reservada enum no se trabajó en clase. Se usa en dos clases del modelo, Categoria y Nivel, y lo que aporta es exactamente lo descrito arriba: una lista cerrada de valores con datos y comportamiento propios. Conceptualmente no introduce nada nuevo frente a lo visto en el curso; una enumeración de Java es una clase con un conjunto fijo de instancias creadas de antemano, y los métodos admiteEdad y esAdyacente son métodos de instancia corrientes. El mismo resultado se podría lograr con una clase normal de constructor privado y constantes estáticas públicas; la enumeración es la forma que el lenguaje ofrece para ese mismo patrón, con la ventaja de que el compilador verifica que no existan otros valores.

10. USO DE JAVA.TIME PARA FECHAS Y HORAS

10.1 El problema

El caso del club está lleno de reglas que dependen de fechas y de horas, no de su representación sino de su comparación. La categoría de un equipo se decide por la edad del jugador, que es la diferencia entre su fecha de nacimiento y la fecha de la inscripción. Una reserva solo se puede crear si la instalación está libre en ese rango de horas, lo que obliga a comparar el inicio y el fin de la reserva nueva contra los de todas las reservas que ya existen. La cobertura mínima de personal se evalúa sobre el horario de atención del club, que es también un rango de horas. Los informes de ventas se piden entre dos fechas.

10.2 La decisión

Los atributos de fecha son de tipo LocalDate y los de hora, de tipo LocalTime, ambos del paquete java.time de la biblioteca estándar de Java. Aparecen en las clases que necesitan fechar o ubicar algo en el tiempo: personas, inscripciones, reservas, partidos, sesiones de entrenamiento, turnos, sanciones, ventas y códigos de descuento.

10.3 Aclaración sobre un tema no visto en clase

Este paquete no se trabajó en clase. Es parte de la biblioteca estándar de Java, de modo que no es una librería externa y no contradice la restricción del enunciado sobre no usar librerías de terceros. De estas dos clases se usan únicamente cuatro cosas, todas ellas métodos corrientes sobre objetos inmutables: isBefore e isAfter para comparar dos instantes, que son las que aparecen en las validaciones; parse para construir un valor a partir del texto que viene en los archivos de datos,

y aparece únicamente dentro del repositorio; y Period.between junto con now en un solo método, Persona.getEdad, para calcular los años cumplidos. No se usan zonas horarias, calendarios alternativos, formateadores ni aritmética de fechas.

10.4 Qué pasaría si no se hubiera hecho así

Con la fecha guardada como texto habría que escribir a mano la comparación: separar la cadena en día, mes y año, convertir cada parte a número y compararlos en orden. Eso hay que repetirlo en cada validación y es donde aparecen los errores difíciles de ver, porque "09/2026" y "9/2026" son cadenas distintas que significan lo mismo. Con la fecha guardada como tres enteros, el problema es el mismo multiplicado por tres atributos. Con horas guardadas como números enteros habría que decidir si se cuentan minutos desde medianoche o se usan dos campos, y el cruce de reservas, que es la regla más delicada del caso, quedaría escrito en términos de esa convención en lugar de en términos del dominio.

10.5 Cómo se guardan en los archivos

En los archivos planos las fechas y las horas se escriben en el formato ISO, 2026-03-15 y 18:30, que es el que producen toString y el que acepta parse sin configuración adicional. Eso mantiene los archivos legibles y editables a mano, que es la razón por la que se eligieron archivos planos.

11. LIMITACIONES ASUMIDAS CONSCIENTEMENTE

11.1 Un afiliado no puede ser de conjunto e individual a la vez

Java no permite herencia múltiple, de modo que una persona que juegue fútbol y además tome clases de tenis necesitaría dos registros distintos en el sistema.

Se dejó así porque es la estructura que se sustentó en la Entrega 1 y porque el enunciado no plantea explícitamente ese caso. Si llegara a hacer falta, hay dos salidas: dejar un solo Afiliado con las dos listas de inscripciones, o modelar los roles como objetos aparte en lugar de como subclases. La segunda es más flexible pero reparte los datos del afiliado en varios lugares.

11.2 La fachada concentra bastante responsabilidad

La clase Club tiene más métodos que cualquier otra del sistema. Es el costo conocido del patrón Fachada. Se mitigó dejando en ella únicamente las operaciones que coordinan a varios objetos, pero si el sistema crece convendría dividirla en varias fachadas por área: una deportiva, una comercial y una administrativa.

UNIVERSIDAD DE LOS ANDES
ISIS-1226 · DISEÑO Y PROGRAMACIÓN ORIENTADA A OBJETOS

PATRONES DE DISEÑO APLICADOS

Sistema de analítica deportiva del Club Deportivo Séneca

Proyecto 1 — Entrega 2

1. CRITERIO DE SELECCIÓN

El diseño aplica cinco patrones. Ninguno se incorporó por completitud: cada uno responde a un problema concreto del Club Deportivo Séneca que ya existía antes de pensar en patrones. Para dejarlo demostrado, la descripción de cada uno termina indicando qué ocurriría si no se hubiera aplicado, que es la única forma de verificar que el patrón hace un trabajo real.

Conviene además aclarar algo: ninguno de estos patrones usa mecanismos que no se hayan visto en el curso. Strategy, Factory Method y DAO son aplicaciones de interfaces y polimorfismo; Template Method es herencia con un método marcado como final; Fachada es simplemente una clase que coordina a otras.

2. STRATEGY

2.1 Dónde está

En tres lugares. Las interfaces ReglasDeporte, con sus implementaciones ReglasFutbol, ReglasBaloncesto y ReglasVoleibol; ReglasDisciplina, con ReglasTenis, ReglasPadel y ReglasTenisMesa; y FormatoCompetencia, con FormatoLiga y FormatoFases.

2.2 El problema que resuelve

El club participa en dos circuitos universitarios que son la misma cosa —una competencia con equipos, partidos y clasificación— pero que se resuelven de forma distinta. El Torneo de los Cerros se juega como liga de todos contra todos, donde cada resultado otorga puntos según el deporte. Los Juegos ASCUN se juegan por fase de grupos seguida de eliminación directa, donde en la segunda fase los puntos dejan de importar y lo que decide es el enfrentamiento.

Lo mismo ocurre con los deportes. Fútbol otorga tres puntos por victoria y uno por empate; baloncesto y voleibol no admiten empate. Fútbol maneja tarjetas, los otros dos no. Voleibol lleva el marcador por sets.

2.3 Cómo funciona en el modelo

La clase Competencia no sabe si es una liga o un torneo por fases: guarda una referencia a su formato y le pide la clasificación. La clase Equipo no sabe de qué deporte es: guarda una referencia a sus reglas y les pregunta cuántos jugadores necesita para presentarse a un partido.

2.4 Qué pasaría sin el patrón

La clase Competencia tendría un atributo de texto llamado formato, y cada método que calculara posiciones empezaría preguntando si el valor es «liga» o «fases». Ese condicional se repetiría en varios lugares y habría que acordarse de actualizarlos todos al agregar un formato nuevo. Lo mismo con el deporte: las clases Equipo y Partido quedarían llenas de condicionales encadenados.

3. FACTORY METHOD

3.1 Dónde está

En el método crearEstadistica(jugador, partido), declarado dentro de las interfaces ReglasDeporte y ReglasDisciplina e implementado por cada juego de reglas.

3.2 El problema que resuelve

Cada deporte produce un tipo distinto de estadística. Quien registra un partido tendría que decidir si construye una EstadisticaFutbol, una EstadisticaBaloncesto o una EstadisticaVoleibol, lo que significa volver a depender del deporte concreto justo donde se quiso evitar esa dependencia.

3.3 Cómo funciona en el modelo

Cada implementación de reglas sabe qué estadística le corresponde y la construye. Quien registra un partido escribe reglas.crearEstadistica(jugador, partido) y recibe el objeto adecuado sin saber de qué tipo es.

3.4 Qué pasaría sin el patrón

El código que registra partidos tendría que decidir con condicionales qué clase instanciar. Agregar un deporte nuevo obligaría a modificar ese código, cuando la idea era que bastara con escribir una clase de reglas nueva.

3.5 Relación con Strategy

Los dos patrones conviven en la misma interfaz y eso no es casualidad: el objeto que sabe las reglas de un deporte es exactamente el que sabe qué estadística produce ese deporte. Separarlos obligaría a mantener dos jerarquías paralelas con la misma cantidad de clases.

4. TEMPLATE METHOD

4.1 Dónde está

En el método liquidar() de la clase Venta y en calcularNuevoPuntaje() de ReglasRaquetaBase.

4.2 El problema que resuelve

Liquidar una venta es siempre el mismo procedimiento, con los mismos pasos y en el mismo orden: sumar los ítems, aplicar el descuento si hay un código, calcular el impuesto sobre la base ya descontada, sumar los valores adicionales y generar los puntos de fidelidad. Lo único que cambia entre la tienda y la cafetería son dos detalles: el porcentaje del impuesto, que es diecinueve por ciento de IVA frente a ocho por ciento de impuesto al consumo, y si la venta admite propina.

4.3 Cómo funciona en el modelo

El método que define el procedimiento está marcado como final en la clase padre y deja dos huecos que cada subclase completa.

```
public final double liquidar( )
{subtotal = calcularSubtotal( );
        descuento = calcularDescuento( );
        double base = subtotal - descuento;
        impuesto = base * getPorcentajeImpuesto( ); // lo resuelve la subclase
        valorAdicional = calcularValorAdicional( base ); // solo la cafeteria lo redefine total = base + impuesto +
        valorAdicional;
        puntosGenerados = generarPuntosFidelidad( );
        liquidada = true;
        return total;
}
```

4.4 Por qué el método es final

Para que ninguna subclase pueda saltarse un paso ni alterar el orden. Una subclase solo tiene permitido completar los dos huecos; el procedimiento en sí no es negociable. Si VentaCafeteria pudiera redefinir liquidar(), nada impediría que calculara el impuesto antes del descuento, que da un resultado distinto.

4.5 Qué pasaría sin el patrón

Cada subclase repetiría los seis pasos completos. Si mañana el club agrega un paso nuevo, por ejemplo un redondeo o un recargo, habría que acordarse de ponerlo en las dos subclases y en el mismo lugar.

5. DAO

5.1 Dónde está

En la interfaz RepositorioClub, con las implementaciones RepositorioArchivosPlanos y RepositorioSerializado.

5.2 El problema que resuelve

El enunciado exige que la información sea persistente y que se guarde en archivos, pero no fija el formato. El sistema necesita que las clases del modelo no sepan nada sobre cómo se almacenan: si lo supieran, cambiar el formato obligaría a tocar todo el modelo.

5.3 Cómo funciona en el modelo

La interfaz declara dos operaciones: guardar un club y cargarlo. Cambiar de un mecanismo a otro es cambiar la línea donde se construye el repositorio. El resto del sistema no se entera.

5.4 Qué pasaría sin el patrón

Las clases del modelo tendrían métodos de lectura y escritura de archivos, mezclando la lógica del negocio con detalles de entrada y salida. Si en el Proyecto 2 se decidiera pasar a una base de datos, habría que modificar todas las clases en lugar de escribir una tercera implementación.

6. FACHADA

6.1 Dónde está

En la clase Club, del paquete logica.

6.2 El problema que resuelve

Varias operaciones del sistema no consisten en llamar a un método sino en coordinar a varios objetos en un orden determinado. Registrar un partido interno, por ejemplo, exige validar que ambos equipos tengan el mínimo de jugadores habilitados, guardar el marcador, descontar un partido a las sanciones que ya estaban vigentes y crear las nuevas que generaron las tarjetas de ese partido, en ese orden exacto.

6.3 Cómo funciona en el modelo

Quien use el sistema pide la operación completa y la fachada se encarga de la coordinación. En la fachada están únicamente las reglas que cruzan varias clases; las que pertenecen a un solo objeto viven en ese objeto.

6.4 Qué pasaría sin el patrón

Cada programa que registre un partido tendría que ejecutar los cuatro pasos por su cuenta y acordarse del orden. Si alguien crea las sanciones antes de descontar las viejas, una tarjeta recibida hoy quedaría cumplida con el partido de hoy, que es justamente el que acaba de jugarse.

7. RESUMEN

Patrón	Dónde	Qué problema del caso resuelve
Strategy	ReglasDeporte, ReglasDisciplina, FormatoCompetencia	Deportes y circuitos con la misma estructura pero reglas distintas.
Factory Method	crearEstadistica en las interfaces de reglas	Cada deporte produce un tipo distinto de estadística.
Template Method	Venta.liquidar y ReglasRaquetaBase.calcularNuevoPuntaje	Procedimiento fijo con dos variaciones entre tienda y cafetería.
DAO	RepositorioClub y sus dos implementaciones	Persistencia en archivos sin acoplar el modelo al formato.
Fachada	La clase Club	Operaciones que coordinan varios objetos en un orden fijo.

UNIVERSIDAD DE LOS ANDES
ISIS-1226 · DISEÑO Y PROGRAMACIÓN ORIENTADA A OBJETOS

MODELO DE PERSISTENCIA Y MANEJO DE ERRORES

Sistema de analítica deportiva del Club Deportivo Séneca

Proyecto 1 — Entrega 2

1. REQUISITOS DE PERSISTENCIA

El enunciado fija cuatro condiciones. La información debe ser persistente; debe almacenarse en archivos, planos o binarios, en el formato que el grupo defina; esos archivos deben estar en una carpeta distinta a la del código fuente; y la persistencia no tiene que hacerse necesariamente en un solo archivo, de modo que hay que diseñar con cuidado cuántos habrá y cómo estarán estructurados.

El sistema implementa dos mecanismos distintos detrás de una misma interfaz. No es redundancia: cada uno resuelve una necesidad que el otro no cubre bien.

2. MECANISMO PRINCIPAL: ARCHIVOS PLANOS

2.1 Un archivo por entidad

Los datos maestros se guardan en siete archivos dentro de la carpeta data, uno por tipo de entidad.

Archivo	Contenido
club.txt	Configuración general: horario de atención y mínimos de personal.
personas.txt	Afiliados, empleados y administrador.
equipos.txt	Equipos de deportes de conjunto.
inscripciones.txt	Vínculo entre jugador y equipo.
academias.txt	Vínculo entre afiliado y disciplina individual.
instalaciones.txt	Canchas y mesas.
productos.txt	Artículos deportivos, bebidas y snacks.

2.2 Por qué varios archivos y no uno solo

Las entidades tienen columnas completamente distintas. Una persona lleva documento, nombre, fecha de nacimiento y credenciales; un producto lleva código, precio y cantidad disponible. En un único archivo, casi todas las filas quedarían con la mayoría de las columnas vacías, y habría que poner un campo de tipo al comienzo de cada línea para saber cómo interpretar el resto.

Con un archivo por entidad, cada uno tiene su propio encabezado y un número fijo de columnas. Eso permite abrirlo, leerlo y editarlo a mano, que es justamente lo que se necesita para preparar los casos de las pruebas de la entrega siguiente.

2.3 Por qué punto y coma y no coma

Varios campos de texto del dominio contienen comas: los nombres de las competencias, las descripciones de los alérgenos, los motivos de una sanción. Si el separador fuera la coma, habría que encerrar esos valores entre comillas y escribir un analizador que distinga las comas separadoras de las comas del contenido. Con punto y coma, una línea se parte con una sola instrucción.

2.4 Estructura de una línea

Las líneas que empiezan por almohadilla son encabezados que documentan las columnas y se ignoran al leer. El resto son registros. Este es el archivo de equipos completo:

```
# codigo;nombre;deporte;categoria;cupoMaximo;activo
EQ-01;Leones FC;Futbol;Mayores;18;true
EQ-02;Halcones FC;Futbol;Mayores;18;true
EQ-03;Titanes BC;Baloncesto;Mayores;12;true
EQ-04;Condores VB;Voleibol;Mayores;14;true
```

2.5 Cómo se guardan las referencias entre entidades

Las referencias se guardan por código y no por objeto. El archivo de inscripciones, por ejemplo, guarda el documento del jugador y el código del equipo, no los objetos completos.

Esto obliga a que el orden de carga importe: primero las personas, después los equipos y por último las inscripciones, que dependen de los dos anteriores. El método que carga el club respeta ese orden y está documentado en el código.

2.6 Cómo se reconstruyen las subclases

Una línea de texto no conserva la clase del objeto que la generó. Por eso los archivos de personas y de productos llevan el tipo en la primera columna: al leer, ese campo decide si se construye un AfiliadoConjunto, un Entrenador o un Administrador.

3. MECANISMO COMPLEMENTARIO: SERIALIZACIÓN BINARIA

3.1 Por qué existe

El modelo del club es un grafo de objetos con referencias cruzadas: un jugador apunta a sus inscripciones, cada inscripción apunta a su equipo, y el equipo apunta de vuelta a la lista de sus inscripciones. Reconstruir ese grafo desde archivos de texto exige resolver las referencias en un segundo recorrido, después de haber creado todos los objetos.

La serialización de Java hace eso sola: guarda el grafo completo y lo reconstruye sin perder ninguna conexión. Para el estado completo del sistema es el mecanismo adecuado.

3.2 Su desventaja, y por eso no es el único

El archivo resultante es frágil. Si cambia la estructura de una clase o la versión de Java, los archivos guardados con la versión anterior dejan de poder leerse. Tampoco se puede abrir para revisarlo: es binario.

Por eso la combinación de los dos mecanismos: los archivos planos para los datos que hay que poder revisar y editar, la serialización para el estado completo.

4. LA INTERFAZ QUE LOS DESACOPLA

Los dos mecanismos implementan RepositorioClub, que declara únicamente tres operaciones: guardar un club, cargarlo y describir el mecanismo. Ninguna clase del modelo conoce las implementaciones concretas.

La consecuencia práctica es que cambiar de un mecanismo a otro es cambiar la línea donde se construye el repositorio. Si en el Proyecto 2 se decidiera pasar a una base de datos, bastaría con escribir una tercera implementación.

5. MANEJO DE ERRORES

5.1 La jerarquía de excepciones

Todos los errores del dominio heredan de ClubException, que extiende Exception. Las nueve excepciones son: CupoLlenoException, cuando un equipo ya alcanzó su cupo máximo.

CategoriaInvalidaException, cuando la edad no corresponde a la categoría.

JugadorSuspendidoException, al intentar alinear a un jugador sancionado.

EquipoIncompletoException, cuando no se alcanza el mínimo de jugadores del deporte.

ReservaNoDisponibleException, cuando la instalación está ocupada o se agotó el cupo simultáneo. NivelIncompatibleException, en un emparejamiento de práctica entre niveles no adyacentes.

InventarioInsuficienteException, cuando no hay unidades suficientes de un producto.

CoberturaMinimaException, cuando un cambio de turno dejaría al club descubierto.

OperacionNoPermitidaException, cuando un rol intenta algo que su perfil no permite.

5.2 Por qué son checked y no unchecked

Son excepciones verificadas, de modo que el compilador obliga a atraparlas o a declararlas. La razón es que no representan errores de programación sino situaciones previsibles del negocio: un equipo incompleto, una cancha ocupada, un cupo lleno. Son resultados posibles de una operación legítima.

Quien llame a esos métodos tiene que decidir qué hacer en cada caso, y el compilador se encarga de que no se le olvide. Una excepción no verificada permitiría ignorar en silencio que una reserva fue rechazada.

5.3 Por qué PersistenciaException está aparte

No hereda de ClubException porque pertenece a otra capa. Su función es encapsular los errores de entrada y salida de bajo nivel — archivo inexistente, disco lleno, archivo corrupto— para que la lógica del negocio no tenga que conocer IOException ni los detalles del mecanismo de archivos.

Esta es la estrategia de capturar una excepción y relanzar otra con más contexto: el repositorio atrapa el error técnico y lanza uno que explica qué se estaba intentando hacer cuando ocurrió.

5.4 Mensajes de las excepciones

Cada excepción se construye con un mensaje que incluye los datos concretos del caso y no solo la descripción genérica del error. En lugar de «el equipo está incompleto», el mensaje dice cuántos jugadores habilitados presenta el equipo y cuántos exige el deporte. Eso permite que el mensaje sea útil tanto para depurar como para mostrárselo a un usuario.

TRAZABILIDAD DE REQUERIMIENTOS

Sistema de analítica deportiva del Club Deportivo Séneca

Proyecto 1 — Entrega 2

1. PROPÓSITO

El análisis de la Entrega 1 identificó veinticinco requerimientos funcionales y veinte restricciones del proyecto. Este documento cruza cada uno contra la clase y el método del diseño que lo implementa.

Sirve para dos cosas. La primera es verificar que el diseño cubre efectivamente lo que se analizó y que no quedó ningún requerimiento sin soporte. La segunda es señalar dónde empezar a leer cuando se quiera revisar o modificar una funcionalidad concreta.

2. REQUERIMIENTOS FUNCIONALES DEL ADMINISTRADOR

RF	Implementación
RF01	Club.agregarTurno(Turno) y la clase Turno.
RF02	Club.aprobarCambioTurno(solicitud, administrador, fecha), apoyado en el método privado contarPersonalCubriendo, que es el que valida el mínimo por rol.
RF03	SugerenciaProducto.aprobar(administrador) y SugerenciaProducto.rechazar(administrador, motivo).
RF04	Club.registrarPersona, Club.agregarEquipo y los métodos setActivo de Afiliado y Equipo para las bajas lógicas.
RF05	Club.getTotalVentas(rubro, desde, hasta) y Venta.getRecibo(), que discrimina los rubros de cada venta.
RF06	Los métodos de consulta de Club sobre instalaciones, reservas, partidos y sesiones, y Producto.getCantidadDisponible().

3. REQUERIMIENTOS DEL AFILIADO DEPORTISTA DE CONJUNTO

RF	Implementación
RF07	Equipo.inscribirJugador(jugador, camiseta, posicion, fecha). Valida en un solo lugar las cuatro reglas: categoría por edad, cupo disponible, camiseta única dentro del equipo y un solo equipo por deporte.
RF08	AfiliadoConjunto.getInscripciones() para los partidos internos y Nomina.puedeAlinear(jugador) para los de competencia.
RF09	AfiliadoConjunto.getSancionesVigentes() y AfiliadoConjunto.estaHabilitado(equipo).
RF10	Afiliado.getEstadisticas() y Estadistica.getResumen(), que cada subclase redefine según el deporte.

4. REQUERIMIENTOS DEL AFILIADO DEPORTISTA INDIVIDUAL

RF	Implementación
RF11	Club.inscribirEnAcademia(afiliado, reglas, nivel, fecha), que crea la InscripcionAcademia correspondiente.

RF12	Club.crearReserva, apoyado en Instalacion.estaLibre y en Club.contarReservasSimultaneas para el cupo por disciplina.
RF13	Club.registrarPartidoPractica(partido, marcadorLocal, marcadorVisitante).
RF14	InscripcionAcademia.getPuntajeRanking() y getHistorialRanking(), este último previsto para las visualizaciones del Proyecto 3.

5. REQUERIMIENTOS DEL ENTRENADOR Y DEL FISIOTERAPEUTA

RF	Implementación
RF15	El constructor de SesionEntrenamiento y Entrenador.agregarSesion(sesion).
RF16	SesionEntrenamiento.convocar(jugador) para convocatorias individuales y convocarEquipo(equipo) para un equipo completo.
RF17	Convocatoria.registrarAsistencia(estado), que valida que el estado sea uno de los tres permitidos.
RF18	Convocatoria.registrarEvaluacion(nota, comentario), que valida el rango de uno a cinco.
RF19	PartidoPractica.setEntrenador y el método validarPuedeJugarse, que omite la regla de niveles cuando participa un entrenador.
RF20	Club.solicitarCambioTurno(codigo, empleado, turno, fecha, motivo).
RF21	Los mismos métodos de solicitud y aprobación. El mínimo de fisioterapeutas se valida por separado del de entrenadores, que es la razón por la que las dos clases están diferenciadas.

6. REQUERIMIENTOS COMUNES A AFILIADOS Y EMPLEADOS

RF	Implementación
RF22	Club.registrarPersona, que rechaza documentos y logins repetidos, y Persona.autenticar(login, password).
RF23	Venta.agregarItem(producto, cantidad) y ProductoCafeteria.getAdvertencia(), que en el caso de Snack construye el aviso de alérgenos.
RF24	Venta.aplicarDescuento(codigo) y CodigoDescuento.getPorcentajeAplicable(esTitular), que distingue el quince por ciento del titular del ocho por ciento compartido.
RF25	La clase SugerenciaProducto y Club.agregarSugerencia(sugerencia).

7. RESTRICCIONES DEL PROYECTO

Las restricciones se hacen cumplir en tres niveles distintos. Algunas por la estructura de clases, que es el nivel más fuerte porque el compilador las verifica; otras por una validación dentro del objeto dueño de la regla; y otras por una validación en la fachada, cuando la regla depende del estado de varios objetos.

7.1 Restricciones garantizadas por la estructura

Restricción	Cómo queda garantizada
El administrador no puede inscribirse a partidos ni reservar instalaciones.	Administrador hereda de Persona y no de Afiliado, y los métodos de reserva exigen un AfiliadoIndividual.
Un afiliado de conjunto no puede reservar instalaciones individuales.	Misma razón: crearReserva exige un AfiliadoIndividual.
Solo el administrador aprueba cambios de turno.	La firma de aprobarCambioTurno exige un Administrador como parámetro.
Solo los jugadores de la nómina juegan los partidos oficiales.	PartidoCompetencia se relaciona con Nomina y no con Equipo.

7.2 Restricciones validadas por el objeto dueño de la regla

Restricción	Dónde se valida
Un jugador pertenece a un único equipo por deporte.	AfiliadoConjunto.yaJuegaDeporte(nombreDeporte)
El número de camiseta es único dentro del equipo.	Equipo.estaCamisetaOcupada(numeroCamiseta)
La edad debe corresponder a la categoría del equipo.	Categoria.admiteEdad(edad)
Mínimo de jugadores habilitados según el deporte.	Equipo.puedeDisputarPartido() y ReglasDeporte.getMinimoJugadores()
Dos amarillas o una roja suspenden al jugador.	EstadisticaFutbol.generaSuspension()
Niveles iguales o adyacentes en partidos de práctica.	AfiliadoIndividual.puedeJugarContra(rival, disciplina)
Capacidad de la instalación según la modalidad.	ReglasDisciplina.esNumeroJugadoresValido(numeroJugadores)
No se despachan bebidas calientes en instalaciones techadas.	Bebida.puedeEntregarseEnInstalacionTechada() e Instalacion.admiteBebidaCaliente()
Asistencia solo de jugadores convocados.	SesionEntrenamiento.buscarConvocatoria, que lanza excepción si el jugador no fue convocado.
Un empleado no puede tener dos solicitudes pendientes.	Empleado.tieneSolicitudPendiente()
IVA del diecinueve por ciento en tienda.	VentaTienda.getPorcentajeImpuesto()
Impuesto al consumo del ocho por ciento en cafetería.	VentaCafeteria.getPorcentajeImpuesto()

7.3 Restricciones validadas en la fachada

Son las que no se pueden decidir mirando un solo objeto.

Restricción	Dónde se valida
Mínimo de dos entrenadores y un fisioterapeuta durante el horario de atención.	Club.aprobarCambioTurno, que simula el cambio y cuenta todo el personal del día antes de aplicarlo.
Máximo de instalaciones simultáneas por disciplina.	Club.contarReservasSimultaneas
Nómina mínima y categoría habilitada para inscribirse en una competencia.	Competencia.inscribirEquipo

7.4 Restricciones técnicas

La persistencia en una carpeta distinta a la del código se cumple así: RepositorioArchivosPlanos no trae ninguna ruta escrita dentro de la clase, sino que la recibe como parámetro de su constructor. Quien lo construye le entrega la carpeta data, que está al mismo nivel que src y por fuera de ella, de modo que ningún archivo de datos queda dentro del código fuente. Dejar la ruta en el constructor y no fija en la clase es lo que permite que las pruebas de la entrega siguiente apunten a una carpeta propia sin tocar el
repositorio. El login y la contraseña para todo usuario están en Persona, que es la raíz de la jerarquía, de modo que ninguna clase de usuario puede existir sin ellos. La aplicación está construida en Java, sin librerías externas.

8. COBERTURA

Los veinticinco requerimientos funcionales y las veinte restricciones tienen una implementación identificada en el diseño. No hay requerimientos sin soporte ni clases del modelo que no respondan a algún requerimiento o restricción del análisis.

CAMBIOS RESPECTO A LA ENTREGA 1

Sistema de analítica deportiva del Club Deportivo Séneca

Proyecto 1 — Entrega 2

1. POR QUÉ HUBO CAMBIOS

El enunciado establece que el trabajo de cada entrega parte de la anterior y que, si es necesario, pueden hacerse cambios para mejorar o corregir decisiones tomadas en etapas previas. Este documento deja constancia de los nueve cambios que se hicieron al modelo de dominio y del motivo de cada uno.

Los tres primeros responden a la retroalimentación recibida sobre la Entrega 1. Los seis restantes surgieron al pasar del análisis al diseño, cuando el modelo tuvo que soportar reglas que en el papel se podían enunciar pero que la estructura no permitía aplicar.

2. CORRECCIONES SEÑALADAS EN LA RETROALIMENTACIÓN

2.1 La estadística no conectaba con el afiliado individual

Lo que había: una única clase EstadisticaJugador, con los campos valorPrincipal, valorSecundario y tarjetas, relacionada solo con AfiliadoConjunto.

El problema: los partidos de práctica de las academias individuales también producen estadísticas, porque el enunciado pide registrar sets, games, aces y errores no forzados. Con esa estructura no había dónde guardarlos.

Lo que se hizo: Estadistica pasó a ser abstracta con cinco subclases, una por tipo de deporte, y la relación apunta ahora a Afiliado, que es la clase padre. Como PartidoPractica también es un Partido, los dos casos quedan cubiertos sin duplicar relaciones.

De paso se corrigió otro defecto: los campos valorPrincipal y valorSecundario no decían qué medían. Ahora cada deporte nombra lo suyo.

2.2 Había datos que eran atributo y relación a la vez

El problema: varias clases guardaban como atributo de texto una información que la relación ya expresaba, de modo que el mismo dato quedaba registrado dos veces y podía quedar inconsistente.

Lo que se eliminó:

Partido.tipo, porque el tipo de partido lo determina la subclase concreta y su relación con la competencia. Venta.tipo, porque lo determina si es una VentaTienda o una VentaCafeteria.

ProductoCafeteria.tipoProducto, que se resolvió separando la clase en Bebida y Snack.

Equipo.deporte, que pasó a ser la relación con el objeto de reglas del deporte.

Instalacion.tipo, que pasó a ser la relación con las reglas de la disciplina.

AfiliadoIndividual.nivel, que se movió a InscripcionAcademia, donde sí corresponde.

2.3 Las relaciones se cruzaban demasiado

Lo que se hizo: el diagrama se reorganizó en bloques por paquete, con las jerarquías dispuestas de arriba hacia abajo dentro de cada bloque y las líneas ruteadas de forma ortogonal, evitando que pasen por encima de las cajas. Se agregó además un recuadro de convenciones que explica cómo leer el diagrama.

3. CAMBIOS SURGIDOS AL PASAR AL DISEÑO

3.1 El bloque horario de la reserva

Lo que había: Reserva.bloqueHorario como una cadena de texto.

El problema: para detectar si dos reservas se cruzan hay que comparar franjas horarias. Con un texto habría que interpretar la cadena en cada validación y el formato quedaría a merced de quien escriba el dato.

Lo que se hizo: dos campos horaInicio y horaFin de tipo LocalTime. La detección de cruces es ahora una comparación directa.

3.2 La cardinalidad entre turno y solicitud

Lo que había: Turno uno a SolicitudCambioTurno cero o uno.

El problema: con esa cardinalidad, un turno cuya solicitud fue rechazada no se podría volver a solicitar nunca, lo cual no tiene sentido. *Lo que se hizo:* la cardinalidad pasó a cero o muchas. La regla de que un empleado no puede tener más de una solicitud pendiente se valida en el método tieneSolicitudPendiente, que es donde corresponde.

3.3 La cardinalidad del aprobador

Lo que había: Administrador uno a SolicitudCambioTurno.

El problema: esa cardinalidad obliga a que toda solicitud tenga aprobador desde el momento en que se crea, lo que contradice el estado pendiente.

Lo que se hizo: pasó a cero o uno.

3.4 Clases nuevas

Se agregaron tres conceptos que el análisis había enunciado como reglas pero que el modelo no podía representar.

Nomina representa la inscripción de un equipo en una competencia. Sin ella no hay forma de expresar que solo los jugadores inscritos en ese torneo pueden disputar sus partidos oficiales.

FormatoCompetencia, con las implementaciones FormatoLiga y FormatoFases, permite que el Torneo de los Cerros y los Juegos ASCUN calculen su clasificación de forma distinta sin que la clase Competencia tenga que distinguirlos.

Sancion ya existía en el análisis pero estaba relacionada con el equipo y no con el jugador, de modo que no se podía consultar si un afiliado estaba habilitado. Ahora se relaciona con ambos.

3.5 Categoría y Nivel pasaron a ser enumeraciones

El problema: de la categoría depende un rango de edad y del nivel depende la regla de adyacencia. Con texto libre, esos rangos habría que repetirlos en cada validación y un error de escritura pasaría inadvertido.

Lo que se hizo: Categoria lleva su rango y responde si admite una edad; Nivel lleva un orden y responde si otro nivel es adyacente.

3.6 Atributos que exigían los requerimientos

Varios requerimientos de la Entrega 1 no tenían soporte en el modelo. Se agregaron:

Afiliado.puntosFidelidad, para la regla del dos por ciento del valor de cada compra.

Club.maximoInstalacionesSimultaneas, para el cupo por disciplina exigido en el RF12.

Venta.impuesto y Venta.propina, para la discriminación de rubros del RF05.

Producto.cantidadDisponible, para el control de inventario del RF06.

Competencia.categoriaHabilitada y Competencia.nominaMinima, para los requisitos de inscripción.

Partido.institucionRival y Partido.escenario, para los partidos de competencia.

4. LO QUE NO CAMBIÓ

Se conservó la jerarquía de Persona tal como se sustentó en la Entrega 1, incluida la decisión de que el administrador herede directamente de ella y no de Afiliado. Se conservaron también los siete bloques temáticos del diagrama original, a los que se sumaron cinco nuevos por las clases que se agregaron.

Se mantuvo la separación entre AfiliadoConjunto y AfiliadoIndividual, pese a que implica que una persona que practique las dos modalidades necesite dos registros. Es una limitación conocida y está documentada en el documento de decisiones de diseño.

UNIVERSIDAD DE LOS ANDES
ISIS-1226 · DISEÑO Y PROGRAMACIÓN ORIENTADA A OBJETOS

MANUAL DE INSTALACIÓN Y EJECUCIÓN

Sistema de analítica deportiva del Club Deportivo Séneca

Proyecto 1 — Entrega 2

1. REQUISITOS

El proyecto necesita únicamente un JDK 8 o superior y Eclipse. No usa librerías externas ni requiere configuración adicional.

2. IMPORTAR EL PROYECTO EN ECLIPSE

1. Abrir Eclipse y seleccionar *File*, luego *Import*.
2. Escoger *General* y dentro de esa categoría *Existing Projects into Workspace*.
3. En *Select root directory*, buscar y seleccionar la carpeta ClubDeportivoSeneca.
4. Verificar que el proyecto aparezca marcado en la lista y dar *Finish*.

El proyecto trae sus archivos .project y .classpath, de modo que Eclipse reconoce la carpeta src como fuente y bin como

destino de las clases compiladas.

3. COMPILAR DESDE LA LÍNEA DE COMANDOS

La forma recomendada es importar el proyecto en Eclipse, que compila por su cuenta. Si se quiere verificar la compilación sin Eclipse, el comando no es el mismo en todos los sistemas, porque lo que cambia es la forma de listar los archivos .java de todos los paquetes. Desde la carpeta ClubDeportivoSeneca:

3.1 Windows, en PowerShell

```
javac -encoding UTF-8 -d bin (Get-ChildItem -Recurse -Filter *.java src).FullName
```

3.2 Windows, en el símbolo del sistema

```
dir /s /b src\*.java > fuentes.txt
javac -encoding UTF-8 -d bin @fuentes.txt
```

3.3 Linux o macOS

```
javac -encoding UTF-8 -d bin $(find src -name "*.java")
```

3.4 Por qué el comando cambia

La parte entre paréntesis o entre $( ) no es de Java sino de la terminal: es la instrucción que recorre las carpetas y produce la lista de archivos que hay que compilar. La sintaxis $(find ...) pertenece a las terminales de Linux y macOS y falla en PowerShell, que usa Get-ChildItem.

La forma del símbolo del sistema es la portable y la que conviene usar si hay dudas: javac acepta un archivo que contenga la lista de fuentes, indicado con arroba, de modo que cada sistema solo tiene que saber cómo generar ese archivo.

En cualquiera de los tres casos el comando debe terminar sin salida, lo que indica que no hubo errores. La opción -encoding UTF-8 se incluye para que los comentarios del código se lean igual en cualquier equipo.

4. ESTRUCTURA DE CARPETAS

Carpeta	Contenido
src	Código fuente, organizado en paquetes.
data	Archivos de datos. Está fuera de la carpeta del código, como exige el enunciado.
bin	Clases compiladas.
doc	Documentación del proyecto.

5. SOBRE LA EJECUCIÓN

Esta entrega no incluye ninguna clase con método main. Los programas de consola y las pruebas unitarias corresponden a la entrega siguiente del proyecto, de modo que lo que se entrega aquí es el modelo completo para su revisión.
El proyecto compila sin errores, que es la verificación disponible en esta etapa.

6. LOS ARCHIVOS DE DATOS

La carpeta data contiene siete archivos de ejemplo. No son pruebas: su propósito es mostrar el formato de persistencia y servir de punto de partida para las pruebas de la entrega siguiente.

Son archivos de texto plano separados por punto y coma. Las líneas que empiezan por almohadilla son encabezados que documentan las columnas y el sistema las ignora al leer. Se pueden abrir y editar con cualquier editor de texto.

Archivo	Contenido
club.txt	Configuración general del club.
personas.txt	Afiliados, empleados y administrador.

equipos.txt	Equipos de deportes de conjunto.
inscripciones.txt	Vínculo entre jugador y equipo.
academias.txt	Vínculo entre afiliado y disciplina.
instalaciones.txt	Canchas y mesas.
productos.txt	Artículos, bebidas y snacks.

Al cargar, el orden importa: primero las personas, después los equipos y por último las inscripciones, que dependen de los dos anteriores. Si se editan los archivos a mano, hay que respetar que los códigos referenciados existan.

7. DÓNDE EMPEZAR A LEER EL CÓDIGO

Para entender el sistema, el recorrido recomendado es este:

1. modelo.personas.Persona y sus subclases, que son la base de todo lo demás.
2. reglas.ReglasDeporte y una de sus implementaciones, para ver cómo se separan las reglas de cada deporte. 3. modelo.deportes.Equipo, en particular el método inscribirJugador, que concentra cuatro validaciones. 4. modelo.tienda.Venta, para ver el método plantilla liquidar.
5. logica.Club, que es la fachada y coordina las operaciones que cruzan varias clases.
6. persistencia.RepositorioClub y sus dos implementaciones.

UNIVERSIDAD DE LOS ANDES
ISIS-1226 · DISEÑO Y PROGRAMACIÓN ORIENTADA A OBJETOS

ANEXO DE USO DE INTELIGENCIA ARTIFICIAL GENERATIVA

Sistema de analítica deportiva del Club Deportivo Séneca
Proyecto 1 — Entrega 2

1. ALCANCE DEL USO

El enunciado exige documentar el uso de herramientas de inteligencia artificial generativa, incluyendo la colección de prompts empleados. Esta sección recoge el uso efectivo que hizo el grupo durante la Entrega 2, antes de subir la entrega al repositorio.

2. HERRAMIENTAS UTILIZADAS

Herramienta	Uso dado	Entregable sobre el que se usó
Claude (Anthropic)	Sugerir y justificar el uso de Map<String, X> (LinkedHashMap) para buscar personas por documento y equipos por código sin recorrer listas ni admitir duplicados.	Club.java, CatalogoReglas.java
Claude (Anthropic)	Explicar el uso de enum frente a una clase con constructor privado y constantes estáticas, como alternativa para representar Categoria y Nivel.	Categoria.java, Nivel.java
Claude (Anthropic)	Justificar el uso de java.time (LocalDate, LocalTime) como biblioteca estándar y delimitar los métodos usados (isBefore, isAfter, parse, now, Period.between).	Clases del modelo que manejan fechas y horas (persistencia, modelo.personas, entre otras)
Claude (Anthropic)	Aplicar correcciones puntuales a partir del informe de revisión de código: documentar enum y java.time, corregir el comando de compilación del README para PowerShell y precisar que el repositorio recibe la ruta de datos por constructor.	README.txt, documento de trazabilidad de requerimientos
Claude (Anthropic)	Contrastar los diagramas de secuencia contra el código método por método y revisar el diagrama de clases para detectar asociaciones de Club no representadas.	Diagrama de clases, diagramas de secuencia

3. COLECCIÓN DE PROMPTS

Transcribir aquí los prompts efectivamente utilizados, agrupados por entregable.

3.1 Diseño e implementación

• El Club tiene que poder buscar una persona por su documento y un equipo por su código sin recorrer una lista entera cada vez. ¿Qué me recomiendas usar y por qué?

• ¿Qué es exactamente un Map en Java y en qué se diferencia de una lista? ¿Y qué hace distinto un LinkedHashMap? Explícamelo sencillo porque no lo hemos visto en clase.

• La categoría de un equipo y el nivel de un afiliado son listas cerradas de valores de las que dependen validaciones. ¿Me conviene guardarlas como texto o como enum? Explícame qué es un enum y qué pasaría si lo hiciera con texto.

• ¿Puedo usar LocalDate y LocalTime en un proyecto donde no se permiten librerías externas? Dime qué hace cada método que necesite y por qué no me sirve guardar las fechas como texto.

• El comando `javac -d bin $(find src -name "*.java")` me falla en PowerShell. Dame la versión que funcione en Windows y explícame por qué cambia.

3.2 Documentación

• Revisa la ortografía y la redacción de este documento y corrígela donde haga falta.

4. REVISIÓN DEL GRUPO

Describir aquí qué se revisó, qué se corrigió y qué se reescribió del material generado antes de incorporarlo a la entrega.

Durante la Entrega 2 usamos Claude como apoyo y estos son los puntos donde su aporte cambió el resultado.

Elección del Map. El Club tenía que poder buscar una persona por su documento y un equipo por su código sin recorrer una lista entera cada vez, y sin que se repitiera el identificador. Estructuras de diccionario las habíamos visto en un semestre anterior pero en Python, no en Java, entonces le preguntamos a Claude qué nos recomendaba usar y por qué. Nos explicó que un Map en Java es justamente eso, una estructura que guarda pares de clave y valor, que la clave no se puede repetir porque así funciona por construcción, y que si además usábamos LinkedHashMap el orden en el que íbamos agregando los datos se mantenía, cosa que nos servía para que los reportes salieran siempre igual. Con esa explicación decidimos usar Map<String, X>, en concreto LinkedHashMap, y dejamos esa misma justificación como comentario dentro de Club y de CatalogoReglas.

Uso de enum. La categoría de un equipo y el nivel de un afiliado son listas cerradas de valores de las que dependen validaciones, entonces la duda era si guardarlas como texto o como enum, y ahí tampoco teníamos nada visto en clase. Claude nos explicó qué es un enum, que es una clase con un conjunto fijo de instancias que ya vienen creadas de antemano, y que lo mismo se podría hacer con una clase de constructor privado y constantes estáticas pero sin ese mismo control. Nos convenció el enum porque cada valor lleva su propio dato, el rango de edad en Categoria y el orden en Nivel, y porque si alguien escribe mal el valor el compilador no deja compilar, mientras que con texto libre eso se cuela sin que nadie se dé cuenta.

Uso de java.time. También le preguntamos si podíamos usar LocalDate y LocalTime en un proyecto donde no se permiten librerías externas, porque no sabíamos si contaban como tal. Nos explicó que java.time es parte de la biblioteca estándar de Java, entonces no es una librería externa, y nos dijo qué hacía cada método que necesitábamos: isBefore e isAfter para comparar fechas, parse para leerlas desde el archivo, now para la fecha actual y Period.between para sacar una diferencia de tiempo. Eso fue lo que acotamos usar, nada más, porque si guardábamos las fechas como texto no podíamos compararlas ni hacer esas cuentas directamente.

Correcciones a partir del documento de revisión de código. Le entregamos a Claude el informe de hallazgos de la revisión de código para que nos ayudara a aplicar las correcciones, y de los seis hallazgos aplicó cuatro: documentar el enum y java.time, corregir el comando de compilación del README porque tenía sintaxis de Linux y en PowerShell no corría, precisar en la trazabilidad que el repositorio recibe la ruta de datos por constructor y no la tiene fija, y actualizar la división del trabajo del grupo. Los otros dos hallazgos, la falta de clases de consola y el anexo de IA, esos los resolvimos nosotros mismos.

Verificación de los diagramas. Por último, Claude revisó los diagramas de secuencia contra el código método por método y ahí encontró que al diagrama de clases le faltaban cinco asociaciones de Club. Lo que pasaba era que el diagrama se genera leyendo los atributos del código y la lectura de tipos genéricos solo reconocía los de un parámetro, entonces las colecciones que habíamos declarado como Map<String, X> se quedaban sin aparecer. Lo corregimos y el modelo pasó de tener 91 relaciones a tener 96.